Strategic Software Design

Architecting software that lasts

A working method for the decisions that are expensive to reverse — what makes software last,
where design effort actually belongs, and how the same discipline extends to building with an LLM at the keyboard.

Oleksandr Polieno · **Only Devs #128**

Inspired by Eric Evans' (Strategic) Domain-Driven Design and Kent Beck's eXtreme Programming

AGENDA

Nine stops

01	Software design, and what “lasts” means
02	Not all software needs the same rigor
03	Software as a model of the business domain
04	Delivering in small chunks
05	Before we start building
06	Refactoring — the nature of software
07	Code quality
08	Testing strategy
09	The same principles in the age of LLMs

01 — DEFINITIONS

Software design, and what "lasts" actually means

Software design is the set of decisions that shape how a system can change over time. "Lasts" doesn’t mean untouched — it means still cheap to understand, extend, and correct years in. Here’s what it looks like when that fails:

FAILS WHEN

The technology goes obsolete

The stack, framework, or platform it was built on stops being viable long before the business logic does.

FAILS WHEN

It becomes a big ball of mud

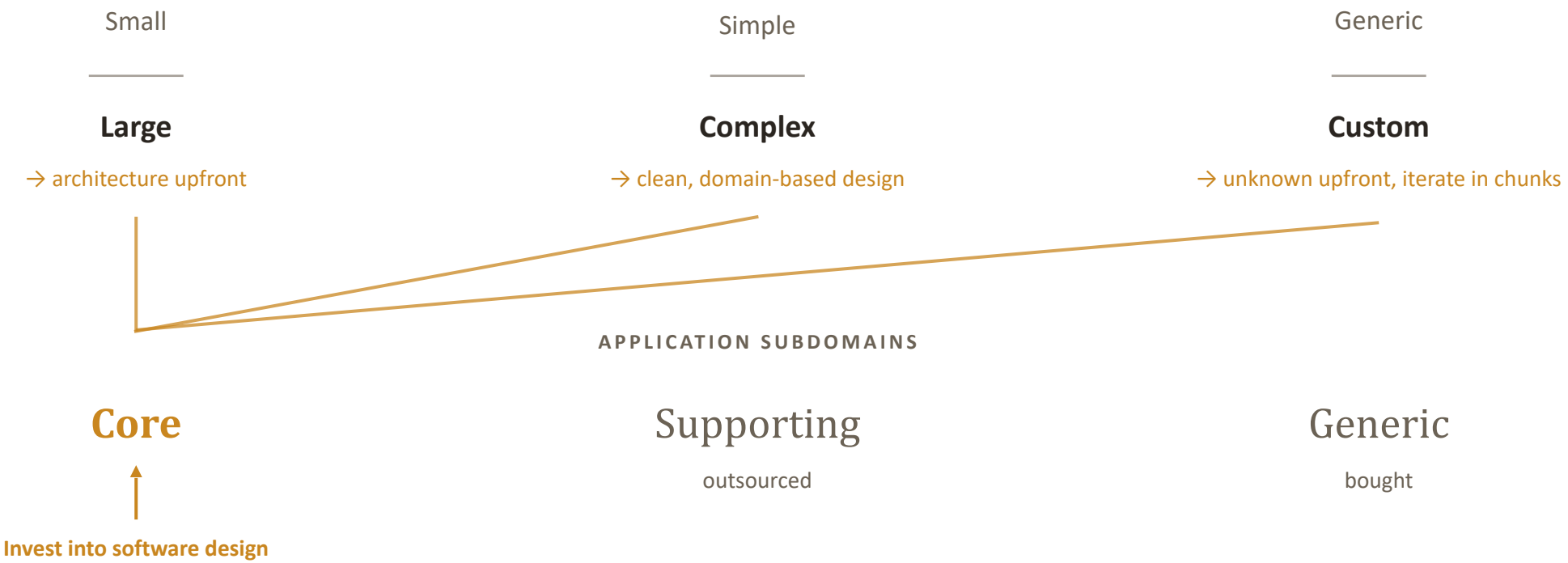
Nobody can extend or maintain it efficiently anymore — every change is a small act of archaeology.

FAILS WHEN

It outgrows its assumptions

It wasn’t designed for the load, data volume, or complexity it eventually had to carry — and more, besides these three.

Some software needs good design more than other software



Good software design matters most exactly where a system is **not simple, not small, and not generic**.

03 — THE CORE IDEA

Software is a model of the business domain

If the code is a model of the business, then the business should shape the software — and even the technology choices made to support it — rather than the other way around. A domain with strict consistency needs shouldn’t be forced into an architecture optimized for something else.

That domain knowledge doesn’t come from a spec document; it comes from domain experts, in ongoing conversation with the people building the system.

WHY IT TAKES BOTH SIDES

Neither side has the whole picture alone

Domain experts usually lack software design experience; developers usually lack the business’s domain experience. Neither gap closes by itself — only by working the model together, continuously.

Which is also why design isn’t only the shape a system ends up with — it’s the ongoing process of building it.

04 — IN PRACTICE

Build in small chunks, not in one long phase

The model can’t be gotten right on paper. Build a thin slice, put it in front of the domain, get feedback, adjust — continuously, not as a phase that ends. Neither side corrects the other from a spec; they correct each other from something real.

PLAN EXTENSIVELY, THEN BUILD



One long phase before the first line of code — the model is assumed correct before anyone’s used it.

BUILD IN SMALL CHUNKS, FROM DAY ONE



Specification and building interleave from the start — the model is corrected continuously, against real use.

05 — BEFORE THE FIRST LINE OF CODE

Some decisions are expensive to change later

Architecture is exactly that set of decisions: the structural choices a whole codebase grows around, which is why getting one wrong doesn't show up on day one — it shows up in every day after.

EXPENSIVE TO REVERSE

Architecture

The shape a system is built around — layers, boundaries, where the domain lives.

EXPENSIVE TO REVERSE

Culture

How the team makes decisions, reviews code, and treats quality — harder to see, and often harder to fix.

Architecture itself depends on the platform — frontend, backend, and mobile each carry their own shapes. For a backend, for example, the choice is typically between:

Layered

Clear boundaries between presentation, application, and data — each tier calling only the one beneath it.

Clean / hexagonal

Ports & adapters decouple the domain from infrastructure — swap a database or UI without touching business rules.

→ *Whichever shape looks right, defer the decision as long as you honestly can — commit only once the domain has taught you enough to be worth locking in.*

Software is built to change, continuously

It evolves while it’s iteratively being built, when it’s extended with new capability, when it’s refactored to keep pace with what’s been learned, and when its dependencies are swapped out from under it. None of these are exceptions to normal development — they are normal development.

CONSEQUENCE

Maintainability is a success requirement

Not a nice-to-have next to features and deadlines — a system that can’t absorb this kind of continuous change hasn’t actually shipped something that lasts.

07 — WHAT MAINTAINABILITY IS MADE OF

Code quality

Maintainability starts with everyday discipline about how the code itself is put together — the habits that keep change local instead of cascading.

STRUCTURE

Low coupling, high cohesion

Enforced with dependency injection and clear module boundaries — the property every architecture shape is really trying to buy.

HABITS

SOLID

Five habits for keeping a change to one part of the system from forcing changes everywhere else.

EVERYDAY

Clean code

The ordinary practices that keep a codebase readable by someone other than the person who wrote it.

PATTERN

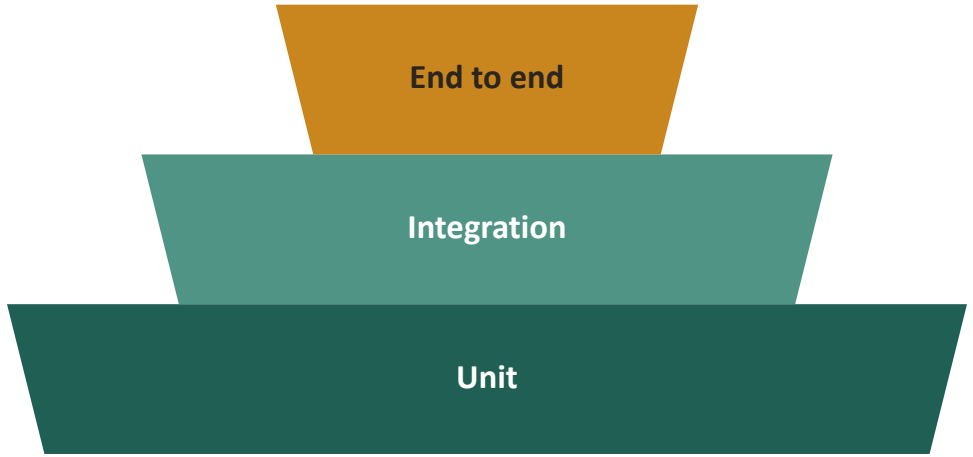
Functional core, imperative shell

Pure business logic at the center, isolated from side effects. Easier to test, easier to reason about.

A testing strategy that matures with you

Start with a small number of integration tests, a handful of unit tests underneath — then let unit coverage grow in as the model stabilizes. Tests give confidence to change the design, and protect the system from regressions while it does. The bar to hold: thorough enough to trust, fast enough it never becomes a reason to skip running it.

More effort
Slower
Higher maintenance



Faster
Low effort
More granularity

The same principles, in the age of LLMs and coding agents

The start looks familiar — it's just written in **.md** files instead of said out loud: **basic information about the domain and the application**, **the architecture**, and **coding guidelines** covering code quality and testing strategy.

What's genuinely different is the iterative process itself — the loop flips who's guiding (steering) whom:



On a non-generic domain, the LLM may not have a memorized solution to fall back on and may not be able to generalize its way to one — so the developer has to supply detailed instructions, and sometimes actual code snippets, the way a domain expert once guided the developer.
Also, there is one more reason for building in chunks: context window limit.
Developer loses a chance to learn about code and domain by writing the code, and have to rely on writing specs, reviews, and interaction with LLM instead.
LLM takes over some of domain expert work.

The model also learns the domain differently than a person does — not by writing the code itself over time, but from the specs it's given and the reviews it gets back. That's precisely why it still needs a human in the loop: to supply the abstract thinking and intuition an LLM doesn't have on its own.

TAKEAWAYS

What to carry forward

01 Save deep design effort for what's genuinely not simple, not small, and not generic — the core domain.

02 Let the business domain shape the software and its technology choices, not the reverse — and get that knowledge straight from domain experts.

03 Decide architecture and culture deliberately before you build — both are expensive to change once the system, and the team, have grown around them.

04 Treat refactoring as normal, not exceptional, and make maintainability — code quality plus a testing strategy that matures with the model — a success requirement.

05 Apply the same discipline with an LLM in the loop: write the decisions down, and expect to guide it in detail on anything that isn't generic.